Thread & Concurrency 2

Operating Systems Ch 4 (Sections 4.5 - 4.7)

Week 5

Today's Schedule

Hour	Content
1st	**Quiz (Beginning)** → Theory Lecture (Part 1)
2nd	Theory Lecture (Part 2)
3rd	Hands-on Lab

Part 1

Implicit Threading

Implicit Threading Overview

Definition: A strategy that delegates thread creation and management to the **compiler and runtime library**

Why is it needed?

- Applications with hundreds to thousands of threads have emerged
- Guaranteeing **correctness** of multithreaded programming is extremely difficult
- Direct thread management by programmers leads to exploding complexity

Core idea: Developers only **identify parallelizable tasks** → library handles thread mapping

Five main techniques:

1. Thread Pools
2. Fork-Join Parallelism
3. OpenMP
4. Grand Central Dispatch (GCD)
5. Intel Thread Building Blocks (TBB)

Implicit Threading: Task vs Thread

Task-based approach

- Tasks are typically written as **functions**
- The runtime library maps tasks to separate threads
- Generally utilizes the **Many-to-Many model**
- Developers do not need to worry about thread count, creation timing, or scheduling

```
┌──────────────────────────────────┐
│        Application Code          │
│   (identify parallel tasks)      │
├──────────────────────────────────┤
│    Compiler / Runtime Library    │
│  (create threads, assign tasks,  │
│   manage scheduling)             │
├──────────────────────────────────┤
│        Operating System          │
│   (kernel thread management)     │
└──────────────────────────────────┘
```

Thread Pools – Concept

Problem scenario (web server example):

- What if a new thread is created for every request?
  - Thread creation takes time → destroyed after completion (wasteful)
  - **No limit** on the number of concurrent threads → risk of system resource exhaustion

Thread Pool solution:

- **Pre-create a fixed number of threads** at startup and keep them waiting in the pool
- When a request arrives, **wake an idle thread** from the pool to handle the task
- If no idle threads are available → task **waits in a queue**
- After task completion, the thread is **returned** to the pool for reuse

```
Request ──→ [ Task Queue ] ──→ ┌─ Thread 1 (busy) ─┐
Request ──→ [ Task Queue ] ──→ │  Thread 2 (idle)   │ ──→ Execute
Request ──→ [ Task Queue ] ──→ │  Thread 3 (busy)   │
                               │  Thread 4 (idle)   │
                               └─ Thread Pool ──────┘
```

Thread Pools – Benefits

Three textbook benefits (Silberschatz, p.178):

1. **Speed**: Reusing existing threads → eliminates wait time for new thread creation
2. **Resource bound**: **Bounds** the number of concurrent threads → prevents system overload
3. **Separation of execution and creation**: Separates task execution from creation mechanism
   - Enables various strategies such as delayed execution, periodic execution, etc.

Determining pool size:

- Set **heuristically** based on CPU count, physical memory, expected concurrent requests
- Can be dynamically adjusted (e.g., GCD automatically adjusts based on system load)

Thread Pools – Windows API

Windows provides a thread pool API, used similarly to Thread_Create()

```
// Define a function to be executed as a separate thread
DWORD WINAPI PoolFunction(PVOID Param) {
    /* this function runs as a separate thread. */
}

// Submit work to the thread pool
QueueUserWorkItem(&PoolFunction, NULL, 0);
```

Three parameters of QueueUserWorkItem():

1. `LPTHREAD_START_ROUTINE Function` – function pointer to execute as a separate thread
2. `PVOID Param` – parameter to pass to the function
3. `ULONG Flags` – flags specifying how the thread pool creates/manages threads

Additional utilities: **periodic execution**, **callback on asynchronous I/O completion**, etc.

Thread Pools – Java ExecutorService

Three thread pool architectures provided by the `java.util.concurrent` package:

Factory Method	Description
`newSingleThreadExecutor()`	Creates a pool of size 1
`newFixedThreadPool(int size)`	Creates a fixed-size pool
`newCachedThreadPool()`	Unbounded pool, reuses idle threads

```
import java.util.concurrent.*;

public class ThreadPoolExample {
    public static void main(String[] args) {
        int numTasks = Integer.parseInt(args[0].trim());

        /* Create the thread pool */
        ExecutorService pool = Executors.newCachedThreadPool();
```

Java ExecutorService – Details

ExecutorService interface structure:

```
      Executor (interface)
          │
          ▼
    ExecutorService (interface)
          │
    ┌─────┴─────┐
    │           │
execute()    submit()    shutdown()
(Runnable)   (Callable)  (shutdown request)
```

execute() vs submit():

- `execute(Runnable)` – executes a task with no return value
- `submit(Callable\<T\>)` – returns a `Future\<T\>`, result can be retrieved later

When shutdown() is called:

- Rejects additional task submissions
- Waits for all currently running tasks to complete before terminating

Fork-Join Parallelism

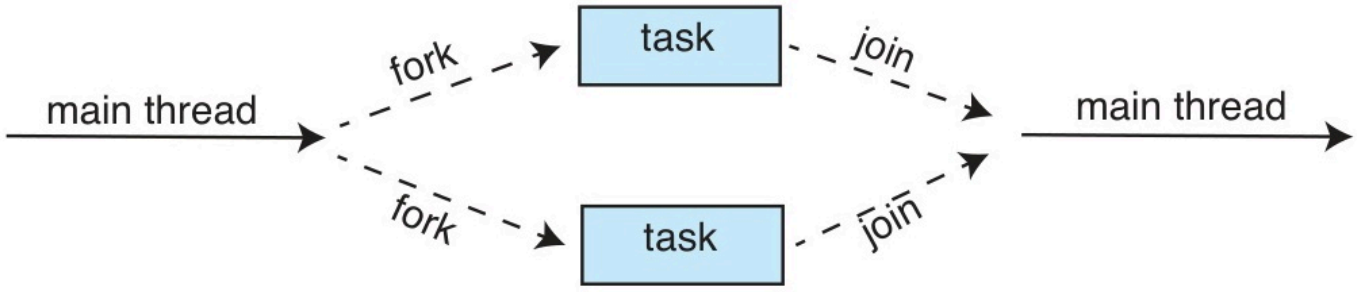
Fork-Join – Concept

Fork-Join model:

- The main (parent) thread **forks** (creates) one or more child tasks
- Child tasks execute in parallel
- Once all children complete, they **join** (merge) and results are combined

From the implicit threading perspective:

- Instead of creating threads directly, **parallel tasks are specified**
- The library determines the number of threads and assigns tasks to them
- A variant of a synchronous thread pool



Fork-Join – Divide-and-Conquer Algorithm

Fork-Join is particularly well-suited for **recursive divide-and-conquer** algorithms

```
Task(problem):
    if problem is small enough:
        solve the problem directly
    else:
        subtask1 = fork(new Task(subset of problem))
        subtask2 = fork(new Task(subset of problem))
        result1 = join(subtask1)
        result2 = join(subtask2)
        return combined results
```

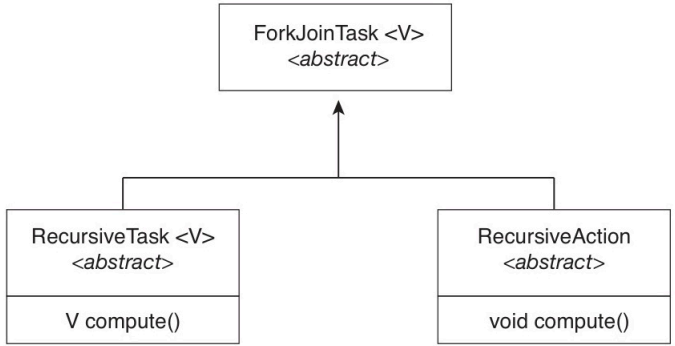
Fork-Join in Java – ForkJoinPool

Fork-Join framework introduced in Java 1.7:

- **ForkJoinPool**: worker thread pool
- **ForkJoinTask<V>**: abstract base class

```
ForkJoinPool pool = new ForkJoinPool();
// array contains the integers to be summed
int[] array = new int[SIZE];
SumTask task = new SumTask(0, SIZE - 1, array);
int sum = pool.invoke(task);
```

Class hierarchy (Figure 4.19):



Fork-Join in Java — SumTask Implementation

Figure 4.18 — Fork-join calculation using the Java API (textbook p.183)

```java
import java.util.concurrent.*;

public class SumTask extends RecursiveTask<Integer> {
    static final int THRESHOLD = 1000;
    private int begin;
    private int end;
    private int[] array;

    public SumTask(int begin, int end, int[] array) {
        this.begin = begin;
        this.end = end;
        this.array = array;
    }

    protected Integer compute() {
        if (end - begin < THRESHOLD) {
            int sum = 0;
            for (int i = begin; i <= end; i++)
                sum += array[i];
            return sum;
        } else {
            int mid = (begin + end) / 2;
```

Fork-Join – Work Stealing

Work Stealing algorithm:

- Each thread maintains its own **task queue**
- When its own queue is empty → **steals tasks from another thread's queue**
- Efficiently handles thousands of tasks with just a few worker threads

Determining THRESHOLD:

- The criterion for deciding whether a problem is "small enough"
- In the textbook example, THRESHOLD = 1000
- In practice, the optimal value should be determined through **timing experiments**

```
Thread 1: [Task A] [Task B] [Task C]
Thread 2: [Task D]  (empty)  (empty) ← steals Task C from Thread 1
Thread 3: [Task E] [Task F]  (empty)
Thread 4:  (empty)  (empty)  (empty) ← steals Task F from Thread 3
```

OpenMP

OpenMP – Overview

OpenMP (Open Multi-Processing):

- A parallel programming API based on **compiler directives** for C, C++, and Fortran
- Supports parallel programming in **shared-memory** environments
- Developers mark **parallel regions** with directives
- The OpenMP runtime library executes those regions in parallel

Basic usage:

```c
#include <omp.h>
#include <stdio.h>

int main(int argc, char *argv[]) {
    /* sequential code */

    #pragma omp parallel
    {
        printf("I am a parallel region.");
    }

    /* sequential code */
```

OpenMP – parallel for

Automatic parallelization of for loops: `#pragma omp parallel for`

```
#pragma omp parallel for
for (i = 0; i < N; i++) {
    c[i] = a[i] + b[i];
}
```

- **Automatically partitions** the operation of adding elements of arrays a and b into c
- OpenMP divides the iteration space among as many threads as available
- Example: N=1000, 4 threads → each thread handles 250 iterations

Execution model:

```
Sequential    #pragma omp parallel for     Sequential
   code    ─→  ┌─ Thread 0: i=0..249  ─┐ ─→  code
               │  Thread 1: i=250..499  │
               │  Thread 2: i=500..749  │
               └─ Thread 3: i=750..999 ─┘
                    (implicit barrier)
```

OpenMP – Additional Features

Manually setting the number of threads:

```
#include <omp.h>
#include <stdio.h>

int main() {
    omp_set_num_threads(4);  // set number of threads to 4

    #pragma omp parallel
    {
        printf("Thread %d of %d\n",
            omp_get_thread_num(),
            omp_get_num_threads());
    }

    return 0;
}
```

Data sharing attributes:

- `shared(var)` – variable shared by all threads
- `private(var)` – each thread gets its own copy of the variable

OpenMP – reduction Example

Parallelizing array summation with reduction:

```c
#include <omp.h>
#include <stdio.h>

int main() {
    int sum = 0;
    int a[1000];

    // Initialize array
    for (int i = 0; i < 1000; i++)
        a[i] = i + 1;

    // reduction: automatically sums partial results from each thread
    #pragma omp parallel for reduction(+:sum)
    for (int i = 0; i < 1000; i++) {
        sum += a[i];
    }

    printf("Sum = %d\n", sum);  // 500500
    return 0;
}
```

- Each thread maintains a **private copy** of sum

Grand Central Dispatch (GCD)

Grand Central Dispatch – Overview

GCD (Grand Central Dispatch):

- Technology developed by Apple for macOS/iOS
- A combination of **runtime library + API + language extensions**
- Similar to OpenMP, automatically manages threading details

Core mechanism: **Dispatch Queue**

- Tasks placed in a dispatch queue are executed by GCD, which **assigns threads from a thread pool**

Two types of Dispatch Queues:

Type	Description
Serial Queue	Dequeues in FIFO order, executes only one task at a time
Concurrent Queue	Dequeues in FIFO order, executes multiple tasks simultaneously

GCD – Serial vs Concurrent Queue

Serial Queue (= Private Dispatch Queue):

- Each process has its own serial queue (**main queue**)
- Developers can create additional serial queues
- Useful for **tasks requiring sequential execution**

Concurrent Queue (= Global Dispatch Queue):

- System-wide queues
- Four **QoS (Quality of Service)** classes:

QoS Class	Description	Example
`QOS_CLASS_USER_INTERACTIVE`	UI/event handling, small tasks	Animations
`QOS_CLASS_USER_INITIATED`	User is waiting for results	Opening file/URL
`QOS_CLASS_UTILITY`	Long-running tasks, immediate result not needed	Data import

GCD – Blocks and Closures

In C/C++/Objective-C: Block – defines a self-contained unit of work using `^{ }` syntax

```
// block definition
^{ printf("I am a block"); }

// create a serial queue and submit work
dispatch_queue_t q =
    dispatch_queue_create("my.queue", NULL);

// use concurrent global queue
dispatch_queue_t gq =
    dispatch_get_global_queue(
        DISPATCH_QUEUE_PRIORITY_DEFAULT, 0);

// submit asynchronous work
dispatch_async(gq, ^{
    printf("Running in background\n");
});
```

In Swift: Closure – same concept as Block, used without `^`

```
let queue = dispatch_get_global_queue(QOS_CLASS_USER_INITIATED, 0)
dispatch_async(queue, { print("I am a closure.") })
```

GCD – Internal Architecture

GCD's Thread Pool Management:

- Internally composed of **POSIX threads**
- GCD **dynamically adjusts** the pool size (expand/shrink)
- Automatically managed based on application demand and system capacity

libdispatch library:

- The implementation of GCD
- Open-sourced by Apple under the **Apache Commons License**
- Ported to other operating systems such as FreeBSD

```
┌──────────────────────────────────────────┐
│ Application  (blocks / closures)         │
├──────────────────────────────────────────┤
│ Dispatch Queues (serial / concurrent)    │
├──────────────────────────────────────────┤
│ libdispatch (GCD runtime)                │
│ - Dynamic thread pool management         │
│ - QoS-based scheduling                   │
├──────────────────────────────────────────┤
```

Intel Thread Building Blocks (TBB)

Intel TBB – Overview

Intel TBB (Threading Building Blocks):

- A **C++ template library** for designing parallel applications
- **No special compiler or language extensions required** (works with library alone)
- Developers **identify parallelizable tasks**, and the TBB task scheduler maps them to threads

TBB Task Scheduler characteristics:

- **Load balancing**: distributes workload evenly among threads
- **Cache aware**: prioritizes tasks that are likely to have data in cache

Key features provided:

- Parallel loop constructs (`parallel_for` , `parallel_reduce`)
- Atomic operations
- Mutual exclusion locking
- Concurrent data structures (hash map, queue, vector)

Intel TBB – parallel_for Example

Serial code:

```
for (int i = 0; i < n; i++) {
    apply(v[i]);
}
```

Converted to TBB parallel_for:

```
#include <tbb/tbb.h>
using namespace tbb;

parallel_for(size_t(0), n, [=](size_t i) {
    apply(v[i]);
});
```

How it works:

- First two parameters: iteration space (0 to n-1)
- Third parameter: **C++ lambda function** `[=](size_t i)` → executed for each i
- TBB splits loop iterations into **chunks**

Intel TBB – parallel_reduce Example

Implementing array summation with parallel_reduce:

```
#include <tbb/tbb.h>
using namespace tbb;

// Using parallel_reduce
int sum = parallel_reduce(
    blocked_range<int>(0, n),  // iteration space
    0,                          // initial value
    [&](const blocked_range<int>& r, int local_sum) {
        for (int i = r.begin(); i < r.end(); i++)
            local_sum += data[i];
        return local_sum;
    },
    [](int x, int y) { return x + y; }  // combine function
);
```

- Task partitioning and scheduling are **automatically handled by the library**
- Maintains load balance using the **work-stealing** algorithm
- Available in both commercial and open-source versions (Windows, Linux, macOS)

Implicit Threading – Comparison of 5 Techniques

Technique	Language/Platform	Approach	Key Feature
Thread Pool	Java, Windows, Android	Pre-create threads in pool	General-purpose, reuse
Fork-Join	Java 1.7+	Divide-and-conquer	Work stealing
OpenMP	C/C++/Fortran	Compiler directives	Simplest syntax
GCD	macOS/iOS (C, Swift)	Dispatch queue	QoS-based
Intel TBB	C++	Template library	Cache aware

Common traits:

- Developers only **identify tasks (parallel work units)**
- Thread creation/management/scheduling is **handled by the library/runtime**
- Improves program correctness and reduces development complexity

Part 2

Threading Issues

Threading Issues Overview

Five key issues to consider in multithreaded programming:

1. **fork() and exec() semantics** – duplicating a multithreaded process
2. **Signal Handling** – delivering signals in a multithreaded environment
3. **Thread Cancellation** – canceling a running thread
4. **Thread-Local Storage (TLS)** – per-thread unique data
5. **Scheduler Activations** – communication between kernel and thread library

fork() and exec() Semantics

Problem: What happens when one thread in a multithreaded process calls fork()?

Two possible behaviors:

1. **Duplicate all threads** – new process has all threads from the parent
2. **Duplicate only the calling thread** – new process is single-threaded

```
Parent Process                  Child Process (Option 1)

 ┌──────────────┐
 │ Thread 1     │    fork()     ┌──────────────┐
 │ Thread 2     ├──────────────→│ Thread 1'    │
 │ Thread 3     │               │ Thread 2'    │  All threads duplicated
 └──────────────┘               │ Thread 3'    │
                                └──────────────┘

Parent Process                  Child Process (Option 2)

 ┌──────────────┐
 │ Thread 1     │    fork()     ┌──────────────┐
 │ Thread 2     ├──────────────→│              │
 │ Thread 3     │               │ Thread 2'    │  Only calling thread duplicated
 └──────────────┘               │              │
                                └──────────────┘
```

fork()/exec() – Selection Criteria

Scenario	fork() behavior	Reason
exec() called immediately after fork()	Duplicate only the calling thread	Since exec() replaces the entire process, duplicating other threads is wasteful
Running without exec() after fork()	Duplicate all threads	The child process also needs all threads

When exec() is called:

- **Replaces** the entire process (**including all threads**) with a new program
- Therefore, in the fork() + exec() pattern, duplicating only the calling thread is sufficient

Some UNIX systems: Provide both versions of fork()

Signal Handling – Concepts and Patterns

Signal: A mechanism in UNIX systems to notify a process that a specific event has occurred

Three-step pattern followed by all signals:

1. A signal is **generated** by a specific event
2. The signal is **delivered** to the process
3. The delivered signal is **handled**

Types of signal handlers:

1. **Default signal handler**: default handler executed by the kernel
2. **User-defined signal handler**: custom handler defined by the programmer

Category	Synchronous Signal	Asynchronous Signal
Cause	Triggered by the process itself	Generated externally
Delivered to	The process that performed the action	Delivered to another process

Signal Handling – Multithreaded Environment

Single-threaded: Signals are always delivered to that process (simple)

Multi-threaded: Where should the signal be delivered?

Four options (textbook p.189):

1. Deliver to the **thread that caused the signal** (suitable for synchronous signals)
2. Deliver to **every thread in the process** (e.g., `Ctrl+C`)
3. Deliver to **certain threads only** (threads that have not blocked the signal)
4. Designate a **dedicated signal handler thread**

```
// UNIX: deliver signal to a specific process
kill(pid_t pid, int signal);

// Pthreads: deliver signal to a specific thread
pthread_kill(pthread_t tid, int signal);
```

POSIX: Each thread can specify which signals to accept/block

- Asynchronous signal → delivered to the **first thread** that does not block it

Thread Cancellation – Concept

Thread Cancellation: Terminating a thread **before it has completed**

Usage examples:

- Multiple threads searching a DB → when one finds the result, **cancel the remaining threads**
- Pressing the Stop button in a web browser → **cancel all** image loading threads

The thread to be canceled: called the **target thread**

Two cancellation approaches:

Approach	Description	Safety
Asynchronous cancellation	Immediate termination	Dangerous
Deferred cancellation	Terminates at a cancellation point	Safe

Asynchronous vs Deferred Cancellation

Problems with Asynchronous Cancellation:

- Immediate termination while resources are allocated → **resource leak**
- Cancellation during shared data update → **data inconsistency**
- The OS reclaims system resources but **cannot reclaim all resources**
- Pthreads documentation **discourages its use**

Deferred Cancellation:

- Only sends a "cancellation request" to the thread
- Actual termination occurs only when the target thread reaches a **cancellation point**
- Enables safe resource cleanup

```
Asynchronous:  cancel ──→ [Immediate termination!] (no resource cleanup)

Deferred:      cancel ──→ [continues running] ──→ [cancellation point] ──→ [cleanup then terminate]
```

Thread Cancellation – Pthreads API

Three cancellation modes in Pthreads:

Mode	State	Type
Off	Disabled	–
Deferred (default)	Enabled	Deferred
Asynchronous	Enabled	Asynchronous

```
pthread_t tid;

/* create the thread */
pthread_create(&tid, 0, worker, NULL);

/* cancel the thread */
pthread_cancel(tid);
```

Cancellation Points and Cleanup Handlers

Cancellation Point:

- In deferred cancellation, a point where the thread "can be canceled"
- Most **blocking system calls** in POSIX/standard C library are cancellation points
  - Examples: `read()`, `write()`, `sleep()`, `pthread_join()`, etc.
- The full list can be checked with `man pthreads`

```
while (1) {
    /* do some work for awhile */
    ...

    /* check if there is a cancellation request */
    pthread_testcancel();
}
```

pthread_testcancel():

- Explicitly creates a cancellation point
- If a pending cancellation request exists → thread terminates
- If none → returns normally, execution continues

Thread Cancellation – Java

Java uses a policy **similar to Pthreads' deferred cancellation**

```
Thread worker;
...

/* set the interruption status of the thread */
worker.interrupt();
```

`interrupt()` : Sets the target thread's **interruption status to true**

```
while (!Thread.currentThread().isInterrupted()) {
    // ... do work ...
}
```

isInterrupted(): Returns the thread's interruption status as a boolean

Comparison with Pthreads:

	Pthreads	Java

Thread-Local Storage – Concept

TLS (Thread-Local Storage): A mechanism where each thread maintains its own **copy of data**

Why is it needed?

- Threads fundamentally **share** process data
- However, sometimes threads need **their own unique data**
- Example: Assigning a **unique transaction ID** to each thread in a transaction processing system

TLS vs Local Variables:

	Local Variables	TLS
Lifetime	Only during the function call	**Throughout the thread's entire lifetime**
Visibility	Within the function scope	Accessible from all functions
Similarity	Allocated on the stack	Similar to **static data** (but unique per thread)

TLS – Implementation by Language

TLS support across languages/compilers:

```
// GCC: __thread keyword
static __thread int threadID;

// C11 standard: _Thread_local keyword
_Thread_local int var = 0;
```

```
// Java: ThreadLocal<T> class
ThreadLocal<Integer> value =
    ThreadLocal.withInitial(() → 0);

value.set(42);           // set value for the current thread
int v = value.get();     // get the current thread's value
```

```
// C#: [ThreadStatic] attribute
[ThreadStatic]
static int threadLocalVar;
```

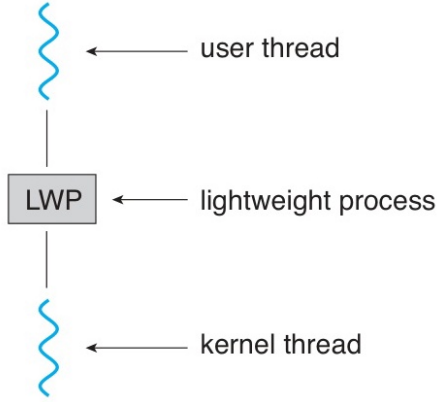
Especially useful in thread pool environments: Each worker thread can maintain unique state even when thread creation is not directly controlled by the developer

Scheduler Activations – LWP and Upcalls

Problem: In Many-to-Many / Two-Level models, **communication** between the kernel and thread library is needed

LWP (Lightweight Process):

- An **intermediate data structure** between user threads and kernel threads
- Appears as a **virtual processor** to the user-level thread library
- Each LWP is **attached to one kernel thread**



Silberschatz, Figure 4.20 – Lightweight process (LWP)

Scheduler Activations – Upcall Mechanism

Upcall: An **event notification** from the kernel to the user-level thread library

Workflow (when a thread blocks):

```
1. Thread A blocks on an I/O request

   ┌──────────────────────────────┐
   │ Kernel: "Thread A is blocking" │ ──→ Upcall
   └──────────────────────────────┘

2. Kernel allocates a new virtual processor (LWP)

3. Thread library executes the upcall handler:
   - Saves the state of the blocking thread
   - Schedules another eligible thread on the new LWP

4. When the blocking event completes:

   ┌──────────────────────────────┐
   │ Kernel: "Thread A is ready"    │ ──→ Upcall
   └──────────────────────────────┘

5. Thread library transitions Thread A back to runnable state
```

Part 3

Operating-System Examples

Windows Threads – Overview

Windows Thread implementation:

- Uses the **One-to-One model** (each user-level thread → kernel thread)
- Threads are created with the Windows API's `CreateThread()`

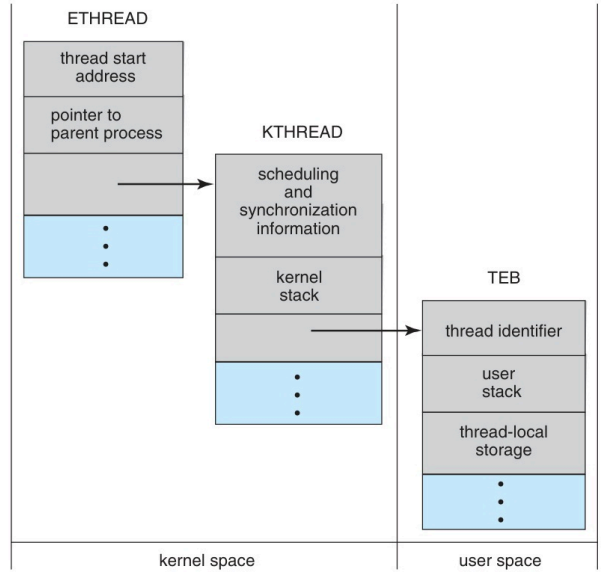
General components of a thread (the thread's **context**):

- Thread ID (unique identifier), Register set (processor state), Program counter
- **User stack** (for user mode) + **Kernel stack** (for kernel mode)
- Private storage area (used by run-time libraries, DLLs)

Three key data structures (Figure 4.21):

Structure	Location	Key Content
ETHREAD	Kernel space	Thread start address, parent process pointer, KTHREAD pointer

Windows Threads – Data Structures (Figure 4.21)



Silberschatz, Figure 4.21 – Windows thread data structures

- ETHREAD, KTHREAD → **accessible only by the kernel** (kernel space)
- TEB → **accessible in user mode** (user space)
- Connection: ETHREAD → KTHREAD → TEB

Linux Threads – Task Concept

Linux's unique approach:

- **Does not distinguish** between process and thread
- All execution flows are called **"tasks"**
- Each task has its own unique **struct task_struct**

Key point: task_struct does not store data directly but contains **pointers to other data structures**

```
┌─ task_struct ──────────────────────────┐
│                                        │
│  *files  ──→ [open file table]         │
│  *signal ──→ [signal handling info]    │
│  *mm     ──→ [memory descriptors]      │
│  *fs     ──→ [filesystem info]         │
│  pid, state, priority, ...             │
│                                        │
└────────────────────────────────────────┘
```

Linux – clone() System Call

clone(): The system call for creating tasks in Linux

- **Flags** determine the **level of sharing** between parent and child

Key flags (Figure 4.22, textbook p.196):

Flag	Shared Resource
`CLONE_FS`	Filesystem information (current directory, etc.)
`CLONE_VM`	Memory space (address space)
`CLONE_SIGHAND`	Signal handlers
`CLONE_FILES`	Open file list (file descriptors)

Linux – fork() vs clone() Comparison

When fork() is called: New task + **copies** all of the parent's data structures

When clone() is called (with sharing flags): New task + **shares** the parent's data structures via pointers

```
fork() – No sharing                 clone() – With sharing flags

 ┌──────────┐  ┌──────────┐          ┌──────────┐  ┌──────────┐
 │ Parent   │  │ Child    │          │ Parent   │  │ Child    │
 │ task_    │  │ task_    │          │ task_    │  │ task_    │
 │ struct   │  │ struct   │          │ struct   │  │ struct   │
 │          │  │          │          │          │  │          │
 │ *mm ─→[A]│  │ *mm ─→[A']│        │ *mm ─┐   │  │ *mm ─┐   │
 │*files→[B]│  │*files→[B']│        │ *files─┤   │  │*files─┤  │
 └──────────┘  └──────────┘          └──────┼───┘  └──────┼───┘
                                             └──→ [SHARED] ←┘
 Different copies                          Same data structures shared
```

Extension of clone() – Containers: By using specific flags to isolate namespaces, containers (Docker, LXC) can be created (Ch 18)

Lab

OpenMP Parallel Programming Lab

Lab: OpenMP Matrix Multiplication Parallelization

```c
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define N 1000

double A[N][N], B[N][N], C[N][N];

void multiply_parallel(int num_threads) {
    omp_set_num_threads(num_threads);
    double start = omp_get_wtime();

    #pragma omp parallel for collapse(2)
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++) {
            C[i][j] = 0.0;
            for (int k = 0; k < N; k++)
                C[i][j] += A[i][k] * B[k][j];
        }

    double end = omp_get_wtime();
    printf("Threads: %d, Time: %.3f sec\n",
           num_threads, end - start);
}
```

Lab: Performance Comparison by Thread Count

```
int main() {
    srand(time(NULL));
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++) {
            A[i][j] = (double)rand() / RAND_MAX;
            B[i][j] = (double)rand() / RAND_MAX;
        }

    multiply_parallel(1);
    multiply_parallel(2);
    multiply_parallel(4);
    multiply_parallel(8);
    return 0;
}
```

Things to observe in the experiment:

- Change in execution time as the number of threads increases
- Comparison between Amdahl's Law and actual speedup
- Performance impact when creating more threads than the number of cores
- Cache effects and memory bandwidth bottleneck

Lab: OpenMP reduction Lab

```c
#include <omp.h>
#include <stdio.h>
#define SIZE 10000000

int main() {
    double *arr = malloc(SIZE * sizeof(double));
    double sum = 0.0;

    // Initialize array
    for (int i = 0; i < SIZE; i++)
        arr[i] = 1.0 / (i + 1);

    // Sequential execution
    double start = omp_get_wtime();
    for (int i = 0; i < SIZE; i++)
        sum += arr[i];
    printf("Sequential sum: %.6f, Time: %.4f sec\n",
           sum, omp_get_wtime() - start);

    // Parallel execution (reduction)
    sum = 0.0;
    start = omp_get_wtime();
    #pragma omp parallel for reduction(+:sum)
    for (int i = 0; i < SIZE; i++)
```

Summary – This Week's Key Points

1. Implicit Threading:

- Delegates thread creation/management to the **compiler and runtime**
- Thread Pool (reuse), Fork-Join (divide-and-conquer), OpenMP (directives)
- GCD (dispatch queue + QoS), Intel TBB (C++ templates)

2. Threading Issues:

- fork(): Duplicate all threads vs. duplicate only the calling thread
- Signal: **4 delivery options** in multithreaded environments
- Cancellation: **Deferred** (safe) vs. Asynchronous (dangerous)
- TLS: Each thread maintains its own unique data
- Scheduler Activations: **LWP + Upcall** mechanism

3. OS Examples:

- Windows: ETHREAD → KTHREAD → TEB (kernel/user space separation)
- Linux: **clone()** flags determine sharing level, no distinction between process/thread – all are **tasks**

Next Week Preview (Week 6)

CPU Scheduling (Ch 5):

- Basic concepts of CPU Scheduling
  - CPU-I/O Burst Cycle
  - CPU Scheduler and Dispatcher
- Scheduling Criteria
  - CPU utilization, throughput, turnaround time, waiting time, response time
- Scheduling Algorithms
  - FCFS (First-Come, First-Served)
  - SJF (Shortest-Job-First)
  - Priority Scheduling
  - Round-Robin (RR)
- Multilevel Queue Scheduling